We actually need to start from the class constructor, because there will be new elements to be added.

Listing 7.9 Java Implementation, LFU Cache constructor

```
LFUCache(int maxSize) {
  this.maxSize = maxSize;
                  #1
  this.hashTable=new ConcurrentHashMap<Key, PriorityQueueNode
>(maxSize);#2
  this.elements = new ConcurrentHeap<Pair<Key, Value>,
Integer>();   #3
  ReentrantReadWriteLock lock = new
ReentrantReadWriteLock();                #4
  this.readLock =
lock.readLock();                                          #5
  this.writeLock = lock.writeLock();
}
```

There are a few noticeable changes, with respect to listing 7.5, besides, of course, switching to Java syntax.

At line 2 and 3, notice how we used a `ConcurrentHashMap` instead of a simple `HashMap`, as maybe you expected, and same goes for the `ConcurrentHeap`[28] class. We want our internal data structures to be synchronized as well, to guarantee atomicity and isolation. Although technically, if we handle the synchronization of `LRUCache`'s methods correctly, there will be no concurrent access al all to its internal fields, so we could also be fine with the regular, single-thread data structures. If we use their concurrent versions, we need to be